Computing the MCI with Glynn's identity

From the DFS to sign sums, exact and approximate

ESIpy

November 2026

1 The problem

For a ring of n atoms with atomic overlap matrices $S_0, \ldots, S_{n-1}$ the MCI is a sum over all orderings of the ring, with atom 0 fixed in front ($m = n - 1$):

$$R = \sum_{\sigma \in S_m} \mathrm{Tr}\left(S_0\, S_{\sigma(1)} S_{\sigma(2)} \cdots S_{\sigma(m)}\right), \qquad \mathrm{MCI} = \tfrac{1}{2}\, 2^{\,n-1} R \;\; \text{(closed shell)}.$$

The DFS visits the $(n-1)!$ orderings one by one. That is $3.6 \cdot 10^5$ for $n = 10$, $6.2 \cdot 10^9$ for $n = 14$ and $1.2 \cdot 10^{17}$ for $n = 20$, which is why the exact MCI stopped at about 14 atoms.

The approximations (mcialg 1–6) keep only the orderings whose consecutive atoms are "close": distance $\le d$, odd distances only, maximum distance exactly d, or a large enough DI. The DFS enumerates those, so its cost is the number of permutations kept.

2 Glynn's identity in one picture

Give every atom a sign $\delta_i = \pm 1$ and add the matrices with those signs, $X = \sum_{i=1}^{m} \delta_i S_i$. Then

$$\boxed{R = \frac{1}{2^{m-1}} \sum_{\delta \in \{\pm 1\}^m,\ \delta_1 = +1} \Big(\prod_i \delta_i\Big)\, \mathrm{Tr}\,(S_0\, X^m)}$$

Why it works. Expanding $X^m = (\delta_1 S_1 + \cdots + \delta_m S_m)^m$ gives every word $S_{i_1} S_{i_2} \cdots S_{i_m}$ of length m, including words that repeat atoms. Each word carries the product of the signs of its letters, times $\prod_i \delta_i$. Summing over all sign choices:

- if an atom appears an *even* number of times (0, 2, …), its sign enters to an odd power overall, and the $+1$ and -1 choices cancel;
- a word of length m in which every one of the m atoms appears an *odd* number of times uses each atom exactly once: it is a permutation, and it survives.

So the sign sum is a filter that throws away every word that is not a permutation. Fixing $\delta_1 = +1$ halves the work by symmetry. The sum has $2^{m-1} = 2^{n-2}$ terms instead of $(n-1)!$, and each term is a matrix power.

Memory. The terms are independent, so they are computed in small batches and summed, and memory does not grow with n. The previous subset-DP approach stored 2^n matrices and exhausted the RAM.

Cost of one term. S_0 has low rank r_0 (about the number of basis functions on the atom). Writing $S_0 = \sum_a w_a u_a v_a^T$ gives $\mathrm{Tr}(S_0 X^m) = \sum_a w_a\, v_a^T X^m u_a$, so we never form X^m: we apply X to a thin block of r_0 vectors. For symmetric AOMs $u^T X^m u = (X^h u) \cdot (X^{m-h} u)$, which needs only $\lceil m/2 \rceil$ products.

3 From DFS approximations to Glynn: adjacency matrices

Every approximation is a rule on consecutive atoms, so it is an *adjacency matrix* A with $A_{uv} = 1$ if the step $u \to v$ is allowed. It is built from the same BFS distances as the DFS:

mcialg	allowed step $u \to v$
1	$\mathrm{dist}(u, v) \le d$
3, 5	$\mathrm{dist}(u, v) \le d$ and odd (5: shortest odd walk)
2, 4	(all steps $\le d$) minus (all steps $\le d - 1$)
6	$\mathrm{DI}(u, v) > \delta$

Instead of the power X^m (which allows any step), walk along A and attach the signs:

$$R_A = \frac{1}{2^{m-1}} \sum_{\delta} \Big(\prod_i \delta_i \Big) \sum_{\substack{\text{walks } 0 \to v_1 \to \cdots \to v_m \to 0 \\ \text{every step allowed by } A}} \delta_{v_1} \cdots \delta_{v_m} \, \mathrm{Tr}(S_0 S_{v_1} \cdots S_{v_m}).$$

The same filter applies: walks that repeat an atom cancel, and what survives are exactly the orderings the DFS would keep. The inner sum is computed step by step, like the DFS but merging all paths that end on the same atom:

$$B_1[v] = A_{0v}\, \delta_v S_0 S_v, \qquad B_{t+1}[v] = \delta_v \Big(\sum_u A_{uv} B_t[u] \Big) S_v, \qquad \text{close with } \sum_v A_{v0}\, \mathrm{Tr}\, B_m[v].$$

Running the same recursion with numbers (all $S = 1$) gives the exact *number of permutations*, which can be compared one-to-one with the DFS count.

Three refinements make it fast:

- **Low-rank blocks:** each AOM is factored at its rank, so a step multiplies small $r \times r$ blocks.
- **Reachability:** at step t only atoms that can still close a cycle are kept, so sparser A means cheaper steps.
- **Two sides:** if every allowed step joins two sides of equal size (odd-only steps on an even ring, i.e. mcialg=3/5), a second sign can be fixed and the number of terms halves to 2^{n-3}. This is detected from A itself, so the same code is general: odd and fused rings simply use 2^{n-2} terms.

For very sparse A ($d = 1, 2$, fullerene cages) there are few permutations and the 2^{n-2} Glynn terms would be wasted. There a pruned search is used: the DFS idea, but abandoning a path as soon as an unused atom cannot be entered and left. C_{60} at $d = 1$ (1090 cycles) takes 10 s.

4 Code

Stripped-down versions of what `esipy/mci.py` does (no batching, low-rank blocks or multiprocessing), so that the three methods can be compared line by line. `S` is the list of AOMs of the ring, atom 0 first.

DFS (current code), exact or approximate. The filter is applied while building each path.

```
def mci_dfs(S, A):
    n = len(S); total = 0.0
    def walk(path, P):
        nonlocal total
        if len(path) == n:
```

```
            if A[path[-1], 0]: # filter here: closing step allowed
                total += np.trace(P)
            return
        for v in range(1, n):
            if v not in path and A[path[-1], v]: # filter here: step allowed, atom unused
                walk(path + [v], P @ S[v])
    walk([0], S[0])
    return total # (n-1)! leaves for the exact MCI
```

Exact MCI with Glynn. No path at all: the sign sum is the filter.

```
def mci_glynn_exact(S):
    m = len(S) - 1; total = 0.0
    for signs in itertools.product([1, -1], repeat=m - 1):
        d = np.array((1,) + signs) # delta_1 = +1
        X = sum(d[i] * S[i + 1] for i in range(m)) # all atoms at once, with signs
        total += np.prod(d) * np.trace(S[0] @ np.linalg.matrix_power(X, m))
        # filter here: words that repeat an atom cancel in this signed sum
    return total / 2 ** (m - 1) # 2^(n-2) terms
```

Approximations with Glynn. The power X^m becomes a walk along the adjacency matrix A; signs still remove repeated atoms, and A removes the forbidden steps.

```
def mci_glynn_adjacency(S, A):
    m = len(S) - 1; total = 0.0
    for signs in itertools.product([1, -1], repeat=m - 1):
        d = np.array((1,) + signs)
        B = [A[0, v] * d[v - 1] * S[0] @ S[v] for v in range(1, m + 1)] # first step 0
            -> v
        for _ in range(m - 1):
            B = [d[v - 1] * sum(A[u, v] * B[u - 1] for u in range(1, m + 1)) @ S[v]
                 for v in range(1, m + 1)] # filter here: only steps u -> v with A[u, v]
                    = 1
        total += np.prod(d) * sum(A[v, 0] * np.trace(B[v - 1]) for v in range(1, m + 1))
        # filter here: repeated atoms cancel in the signed sum, as in the exact case
    return total / 2 ** (m - 1) # same 2^(n-2) terms, cheaper when A is sparse
```

The algorithms are only different matrices A. `dist` is the BFS distance matrix of the ring (`find_node_distances`), `dist_odd` the one that prefers odd walks (`find_node_distances_onlyodd`), with an infinite diagonal.

```
A0 = ~np.eye(n, dtype=bool) # mcialg=0, exact: every step # filter here
A1 = dist <= d # mcialg=1 # filter here
A3 = (dist <= d) & (dist % 2 == 1) # mcialg=3: odd steps only # filter here
A5 = (dist_odd <= d) & (dist_odd % 2 == 1) # mcialg=5 # filter here
A6 = np.abs(DI) > delta # mcialg=6: DI above threshold # filter here

# mcialg=2 and 4: largest step exactly d = (all steps <= d) - (all steps <= d-1)
mci2 = mci_glynn_adjacency(S, dist <= d) - mci_glynn_adjacency(S, dist <= d - 1) #
    filter here
mci4 = mci_glynn_adjacency(S, dist_odd <= d) - mci_glynn_adjacency(S, dist_odd <= d - 1)
     # filter here
```

Every function returns the raw sum R; the MCI is $\frac{1}{2}\,2^{n-1}R$ for closed shells, as in the DFS code. The same recursion with all $S = 1$ counts the permutations, and `mci_dfs(S, A)` gives the same value for the same A.

5 Why exact and approximate cost about the same

With the DFS the cost is the number of permutations, so an approximation that keeps 10^{-5} of them is 10^5 times cheaper. With Glynn the number of terms is 2^{n-2} *whatever* the approximation keeps; only the cost of each term changes. So:

- Glynn makes the *exact* MCI as affordable as the approximations were with the DFS;
- approximations stay useful as *analysis* (distance shells, mcialg=4 fingerprints, which add up to the exact MCI), and they are cheaper by a constant factor, not by orders of magnitude: mcialg=5 at maximum distance has half the terms and cheaper steps, about 5–6 times faster than the exact MCI on annulenes.

The two sides cannot be summed separately ($2^{n/2}$ each): AOMs do not commute, so the two sign sums are tangled in the same product. That shortcut only works for numbers; with real AOMs it is wrong by orders of magnitude (checked).

6 Results (RHF/STO-3G annulenes, meta-Löwdin, 8 cores)

n	permutations exact	Glynn terms	exact	mcialg=5, d_{max}	mcialg=5 / exact
14	$3.1 \cdot 10^9$	4 096	0.23 s	0.09 s	99.99998 %
18	$1.8 \cdot 10^{14}$	65 536	3.4 s	0.6 s	99.9999996 %
22	$2.6 \cdot 10^{19}$	$1.0 \cdot 10^6$	76 s	13 s	99.99999998 %
26	$7.8 \cdot 10^{24}$	$1.7 \cdot 10^7$	32 min	5.2 min	99.9999998 %
28	$5.4 \cdot 10^{27}$	$6.7 \cdot 10^7$	–	24.5 min	–

For comparison the DFS needs 595 s for the exact MCI of $C_{12}H_{12}$, where Glynn needs 0.02 s.

Validation. Every Glynn result is checked against the DFS and against explicit enumeration:

- 900 brute-force checks;
- 979 DFS comparisons, mcialg 0–6 at every d;
- the MCIAPROX dataset (IAO, from the original fchk files).

Values agree to 10^{-13} and the numbers of permutations are identical.

7 Limits

- **Time.** About ×4.7 for every two atoms. Exact MCI: $n \approx 30$ in half a day, $n \approx 32$ in a few days (8 cores). mcialg=5 at maximum distance: about two atoms more.
- **Memory.** Constant (one batch per core).
- **Mathematics.** 2^{n-2} terms cannot be reduced for formulas of this kind, so a whole C_{60} or circumcoronene cage (54–60 atoms) is out of reach exactly; only sparse sums ($d = 1$) or circuits of up to ∼32 atoms are possible.
- **Precision.** The sign sum cancels; rounding is $\sim 10^{-13}$ in absolute value up to $n = 26$. It must be checked for rings whose MCI is tiny.